

Complete Web & SaaS Building Checklist

A practical checklist for building websites, web apps, dashboards, SaaS products and similar projects with AI coding or vibe coding.

This checklist is intentionally practical. The goal is not to turn every project into a giant enterprise system. The goal is to make sure the important things that are commonly forgotten during AI assisted development are checked before launch.

Use only the sections that apply to your project. A simple portfolio website does not need the same infrastructure as a multi user SaaS.

How to Use This Checklist

Do not give the entire checklist to an AI agent and tell it to change everything at once.

A safer workflow is:

1. Understand the project
2. Inspect the existing code
3. Plan the work
4. Implement one phase at a time
5. Test after every important phase
6. Review the final project
7. Deploy only after the final checks

The most important rule is:

Do not change working systems just for the sake of optimization.

Before adding a new library, architecture pattern, database, cache, authentication system or deployment service, check whether the project actually needs it.

PHASE 1 — Understand the Project

Before coding, clearly define what you are building.

Product

- What does the product do?
- Who is it for?
- What is the main problem it solves?
- What is the primary user journey?
- What are the most important features?

- What is required for the first release?
- What is intentionally out of scope?

Users

- Guest user
- Registered user
- Admin
- Other roles if actually required

Features

For every important feature, define:

- Normal flow
- Loading state
- Empty state
- Error state
- Permission denied state
- Mobile behavior
- What happens if the user refreshes?
- What happens if the request fails?

Do this before writing a lot of code. AI agents are much less likely to create inconsistent behavior when the expected behavior is already clear.

Prompt — Project Understanding

Before making any code changes, understand this project completely.

First inspect the existing project structure, technologies, routes, components, APIs, database, authentication, configuration and tests.

Understand what the product is supposed to do and identify the main user flows.

Do not start rewriting the project.

Create a practical implementation plan based on the existing codebase.

Separate the plan into:

1. Required changes
2. Optional improvements
3. Risks
4. Things that should not be changed

Prefer the simplest solution that fits the current project.

Do not introduce unnecessary frameworks, libraries, services or architecture changes.

After the plan is ready, verify that it can be implemented without breaking existing functionality.

PHASE 2 — Project Structure and Code Quality

Before adding more features, make sure the codebase is understandable.

- Clear folder structure
- Components are reasonably reusable
- Business logic is not duplicated everywhere
- API logic is not mixed randomly into UI components
- Environment variables are handled properly
- No obvious temporary files
- No old test files that are no longer relevant
- No unused dependencies where easy to identify
- No duplicate components doing the same job
- No broken imports
- No unnecessary comments explaining obvious code
- Naming is consistent
- Formatting is consistent
- Type checking works if TypeScript is used
- Linting works if configured

Do not perform a massive cleanup just because the code "looks messy". Remove things only when their purpose is understood.

Prompt — Safe Code Cleanup

Perform a careful code quality review of the existing project.

First understand how the project works.

Find obvious dead code, duplicate logic, unused imports, unused dependencies, inconsistent naming and unnecessary temporary files.

Do not rewrite working systems unnecessarily.

Do not change application behavior.

Do not remove code simply because it looks unused unless you can verify that it is actually unused.

Keep the current architecture unless there is a clear technical reason to improve it.

After cleanup, run the existing checks and make sure the project still works exactly as before.

PHASE 3 — UI and UX

A functional application can still feel broken if the user experience is incomplete.

For each page check:

- Desktop layout
- Tablet layout
- Mobile layout
- Navigation
- Loading state
- Empty state
- Error state
- Success state
- Disabled state
- Form validation
- Confirmation for destructive actions
- Clear error messages
- Consistent buttons
- Consistent spacing
- Consistent typography
- Consistent icons
- Keyboard usability
- Focus states
- Touch friendly controls

Do not make every screen visually different. Reuse the same design system and components.

PHASE 4 — Responsive Design

Never test only the desktop screen.

Check common widths such as:

- Small phone
- Large phone
- Tablet
- Laptop
- Large desktop

Also check:

- Navigation does not overflow
- Tables remain usable
- Modals fit on small screens
- Forms fit correctly
- Buttons do not become impossible to tap

- Text does not overlap
- Images do not break the layout
- Horizontal scrolling is intentional only when necessary

PHASE 5 — Database

If the application uses a database:

- Tables or collections are planned
- Relationships are correct
- Important fields have validation
- Unique fields are actually unique
- Important queries have suitable indexes
- Created and updated timestamps exist where useful
- Delete behavior is understood
- Migrations are tracked
- Seed data is separate from production data
- Backups exist for production
- Restore process is tested

For SaaS:

- Every tenant or organization is properly identified
- User data belongs to the correct tenant
- Queries always use the correct tenant scope
- Users cannot access another tenant's records

You do not need a complicated database architecture to have a good SaaS. A clean relational database with proper constraints and authorization is often enough.

Prompt — Database Review

Audit the current database implementation without unnecessarily redesigning it.

Check schema structure, relationships, validation, unique constraints, indexes, migrations and important queries.

Identify only practical improvements that are justified by the current application.

Pay special attention to data ownership and tenant isolation if this is a multi user or multi tenant application.

Do not delete production data.

Do not create destructive migrations.

For every database change, verify its effect on existing features.

After changes, run the relevant tests and verify that the application still works.

PHASE 6 — Authentication

If users can log in, check:

- Registration
- Login
- Logout
- Password reset
- Email verification if needed
- Session expiration
- Session invalidation
- Secure cookies if cookies are used
- Brute force protection
- Rate limiting on login
- Rate limiting on password reset
- Secure password handling
- MFA only if needed

If JWT is used:

- Token expiration
- Secure token handling
- Refresh token strategy where needed
- Token revocation strategy where needed
- Proper signature verification
- No secrets inside tokens
- The server does not blindly trust client supplied identity information

Do not add JWT just because it sounds modern. Use the authentication approach that fits the application.

PHASE 7 — Authorization

Authentication answers:

"Who are you?"

Authorization answers:

"What are you allowed to do?"

This is one of the most commonly forgotten parts of AI generated applications.

Check:

- Every protected page is protected
- Every protected API is protected
- Users can access only their own data
- Admin actions require admin permission
- Role checks happen on the server
- Ownership checks happen on the server
- Tenant checks happen on the server
- Hiding a button is not treated as security

Test negative cases too:

- User tries to access another user's record
- Normal user tries an admin endpoint
- User changes an ID in the request
- User calls an API directly without using the UI

Prompt — Authentication and Authorization

Audit authentication and authorization in the existing application.

First identify the current authentication architecture.

Check login, logout, registration, password reset, session handling, token handling, cookies, role checks and protected routes.

Then verify authorization for every important resource and API.

The frontend must never be treated as the security boundary.

The server must verify identity, permissions and ownership.

For multi tenant systems, verify that one tenant cannot access another tenant's data.

Test both allowed and denied cases.

Do not replace the current authentication system unless there is a real security or architectural reason.

Do not weaken security to make a feature easier to implement.

PHASE 8 — API

If your project has APIs:

- Request validation
- Response validation where useful
- Authentication
- Authorization
- Clear error responses

- Pagination for large lists
- Search limits
- File upload limits
- Request size limits
- Rate limiting
- Timeout handling
- Retry behavior where appropriate
- Idempotency for operations that may be retried
- API documentation for important APIs

Remember that the API must remain secure even when someone completely bypasses the frontend.

PHASE 9 — Input Validation

Treat all external input as untrusted.

Check:

- Forms
- Query parameters
- URL parameters
- Request bodies
- Headers where relevant
- Uploaded files
- Webhooks
- External API responses

Validate:

- Type
- Length
- Allowed values
- Format
- Size

Protect against common problems such as:

- SQL injection
- XSS
- Command injection
- Path traversal
- SSRF where URLs are accepted

Do not rely only on frontend validation. Validate again on the server.

PHASE 10 — File Uploads

If users can upload files:

- File size limit
- File type validation
- Extension validation
- MIME validation where appropriate
- Safe filenames
- Random storage names
- Access control
- Private files stay private
- Signed URLs if appropriate
- Storage limits
- Safe image processing
- Dangerous file types rejected

PHASE 11 — Rate Limiting

Rate limiting is often forgotten until an application is abused.

At minimum consider:

- Login
- Password reset
- OTP
- Email sending
- Search
- Expensive API operations
- File uploads
- Public APIs
- AI generation endpoints

Use stronger limits for actions that can cost money, send emails or consume significant resources.

Do not necessarily rate limit every endpoint aggressively. Choose limits based on the actual product.

PHASE 12 — Caching

Caching can make an application much faster, but incorrect caching can create serious bugs.

For anything you cache, decide:

- What is being cached?
- How long should it live?
- When is it invalidated?
- Can two users safely share the cache?
- Is the data private?
- Is tenant information part of the cache key?

Common layers:

- Browser cache
- CDN cache
- Server cache
- Database or application cache

Never accidentally place private user data in a shared public cache.

For the first version of a small app, simple caching is usually better than building an elaborate caching system.

Prompt — Caching

Review caching in the existing application.

First identify what is currently cached and why.

Do not add caching everywhere.

Only cache data that benefits from caching.

For each new cache, define a sensible TTL and invalidation behavior.

Verify that private user data, authenticated responses and tenant specific data cannot leak through shared caches.

Prefer simple and reliable caching over complicated cache architecture.

Measure or reason about the actual performance benefit before introducing a new caching layer.

PHASE 13 — CDN and Static Assets

If the project is public or serves large assets:

- CDN is used where appropriate
- Images are optimized
- JS and CSS assets are cacheable
- Assets have versioned filenames or suitable cache invalidation

- Compression is enabled
- Large files are not served unnecessarily from the application server
- Private content is not accidentally cached publicly

A CDN is mainly useful for assets and content that benefit from being served close to users. Do not put every authenticated API response behind a public cache.

PHASE 14 — Performance

Before optimizing everything, find actual bottlenecks.

Check:

- Initial page load
- JavaScript bundle size
- Images
- Fonts
- API response time
- Database queries
- Number of network requests
- Unnecessary rerenders
- Large components
- Slow third party services

Useful improvements include:

- Lazy loading
- Code splitting
- Image optimization
- Query optimization
- Pagination
- Caching where useful
- CDN for suitable assets
- Removing unnecessary requests

Do not optimize tiny code paths while ignoring a slow database query or huge image.

PHASE 15 — SEO

For public pages:

- Good page title

- Good meta description
- One clear main heading
- Correct heading structure
- Clean URLs
- Canonical URLs where needed
- robots.txt
- XML sitemap
- Proper 404 page
- Redirect strategy
- Internal links
- Image alt text
- Open Graph metadata
- Social sharing image
- Structured data where genuinely applicable
- Mobile friendly rendering
- Fast public pages
- Search engine indexing checked

For private dashboards, focus on preventing accidental indexing rather than trying to SEO them.

Do not create thousands of low quality AI generated pages just to create more URLs.

PHASE 16 — GEO and AI Search Visibility

There is no magic switch that guarantees visibility in AI search systems.

The practical approach is to make public content clear, useful and easy to understand.

Check:

- Clear company or product identity
- Clear About page
- Clear product descriptions
- Useful documentation
- Useful FAQ pages where appropriate
- Good internal linking
- Structured data where applicable
- Clear author or organization information where useful
- Original useful content
- Consistent naming of products and entities

Focus on real usefulness rather than mass producing generic AI content.

PHASE 17 — Accessibility

At minimum check:

- Keyboard navigation
- Visible focus
- Form labels
- Helpful form errors
- Semantic HTML
- Buttons used for actions
- Links used for navigation
- Images have meaningful alt text when needed
- Color is not the only way information is communicated
- Text remains readable
- Modal dialogs can be closed with keyboard
- Screen reader behavior is not obviously broken

PHASE 18 — Analytics

Only collect the data you actually need.

Check:

- Analytics installed correctly
- Important conversions tracked
- Signup tracked
- Login failure tracked where useful
- Key product actions tracked
- Errors tracked
- UTM parameters handled if using campaigns
- Sensitive information is not accidentally sent to analytics

Analytics should help answer real product questions rather than collect everything possible.

PHASE 19 — Email

If the application sends email:

- Email provider configured

- Domain configuration
- SPF
- DKIM
- DMARC
- Verification emails
- Password reset emails
- Transactional emails
- Failure handling
- Retry handling
- Clear templates
- No sensitive information unnecessarily included in emails

PHASE 20 — Payments

Only applicable if the product charges money.

Check:

- Test payments
- Production payments
- Webhook verification
- Idempotency
- Failed payment handling
- Subscription creation
- Upgrade
- Downgrade
- Cancellation
- Refund
- Invoice
- Tax handling where applicable
- Payment status is verified server side

Never trust the frontend to tell your backend that a payment succeeded.

PHASE 21 — Webhooks

For every webhook:

- Signature verification
- Replay protection where appropriate

- Duplicate event handling
- Retry handling
- Logging
- Failure monitoring
- Safe event processing

Webhook handlers should be able to handle the same event more than once without creating duplicate actions.

PHASE 22 — Secrets and Environment Variables

Never put these directly into source code:

- API keys
- Database passwords
- Payment secrets
- OAuth secrets
- JWT secrets
- Cloud credentials
- Private tokens

Check:

- .env is ignored correctly
- Production secrets are stored securely
- Development and production secrets are separate
- Secrets are not committed to Git
- Leaked keys can be rotated

Also check logs. A secret accidentally printed into an error log is still a leaked secret.

PHASE 23 — Environments

Keep environments separate where practical:

- Local
- Development
- Preview or staging
- Production

Production should not accidentally use:

- Test payment keys

- Development database
- Test email account
- Debug mode
- Fake credentials
- Development URLs

PHASE 24 — Testing

You do not need thousands of tests for every small project.

Focus on important behavior.

- Authentication tests
- Authorization tests
- Main feature tests
- Important API tests
- Form validation tests
- Payment tests if applicable
- Webhook tests if applicable
- Tenant isolation tests if applicable
- Mobile UI checks
- Production build check

Always test failure cases too.

A good test is not only:

"Does the user succeed?"

It is also:

"What happens when the request fails?"

PHASE 25 — Error Handling

A production application needs graceful failure.

Check:

- API errors
- Network errors
- Database errors
- Invalid input

- Expired session
- Missing resource
- Permission denied
- Third party API failure
- Payment failure
- File upload failure

Users should receive understandable messages.

Developers should receive enough logging to debug the actual problem.

Do not show stack traces, secrets or internal implementation details to normal users.

PHASE 26 — Logging and Monitoring

At minimum for production:

- Application logs
- Error tracking
- Basic uptime monitoring
- Important API failure monitoring
- Payment failure monitoring if applicable
- Database health monitoring
- Background job monitoring if applicable

Logs should help answer:

- What happened?
- When?
- Which request?
- Which user or tenant when appropriate?
- What failed?

Avoid logging passwords, tokens and unnecessary sensitive personal data.

PHASE 27 — Backups

If the application stores important data:

- Automated database backups
- Backup retention
- Backup storage separate from the primary database

- Recovery process documented
- Restore test performed

A backup is not fully trusted until you know it can actually be restored.

PHASE 28 — Deployment

Before going live:

- Production environment works
- Domain works
- HTTPS works
- Database migrations are safe
- Environment variables are correct
- Production secrets are correct
- Error monitoring works
- Analytics works
- Emails work
- Payments work if applicable
- Backups work
- Rollback process is understood

PHASE 29 — Legal and Privacy

Keep this as a separate phase.

The exact requirements depend on your country, users, business model, age group, industry and data processing.

Typical items to review include:

- Privacy Policy
- Terms of Service
- Cookie information where relevant
- Refund and cancellation policy where relevant
- Contact information
- Data collection explanation
- Data deletion process
- Account deletion
- Data retention
- Third party services

- Analytics and tracking
- Marketing communication
- User generated content rules where relevant
- Intellectual property
- Open source license obligations
- Taxes and invoicing where relevant

For India based products, check which provisions of applicable Indian privacy and digital regulations apply to your actual business and data processing.

Do not ask an AI to invent legal requirements. Use it to identify what needs review and to keep the implementation consistent with your actual legal documents.

PHASE 30 — Final Pre Launch Checklist

Before launch, ask:

Product

- Does the main user journey work?
- Are important features complete?
- Are error and empty states handled?

Security

- Is authentication working?
- Is authorization enforced on the backend?
- Is rate limiting present where needed?
- Are secrets protected?
- Are user and tenant boundaries safe?
- Are uploads validated?

Performance

- Are pages reasonably fast?
- Are images optimized?
- Are large unnecessary requests removed?
- Are slow queries addressed?

SEO

- Titles

- Meta descriptions
- Canonicals
- Robots
- Sitemap
- Structured data where needed
- Social metadata

Reliability

- Error tracking
- Monitoring
- Backups
- Recovery plan
- Rollback plan

Legal

- Privacy policy
- Terms
- Refund or cancellation information where needed
- Actual product behavior matches published policies

Mobile

- Phone
- Tablet
- Desktop

MASTER PROMPT — BUILD OR IMPROVE A WEB / SAAS PROJECT

Use this when you want an AI coding agent to work on the project.

You are working on an existing production oriented web application.

Do not immediately start changing code.

First understand what the project is supposed to do.

Inspect the actual codebase, project structure, framework, dependencies, routes, components, APIs, database, authentication, authorization, environment configuration, deployment configuration and tests.

Do not guess about the existing architecture.

Your first responsibility is to preserve working functionality.

Before implementation, create a practical plan based on the existing system.

The plan should cover only the areas relevant to this project.

Consider:

- Product requirements
- UI and UX
- Responsive design
- Database
- Authentication
- Authorization
- API
- Input validation
- File uploads
- Rate limiting
- Caching
- CDN and static assets
- Performance
- SEO
- GEO or AI search visibility
- Accessibility
- Analytics
- Email
- Payments
- Webhooks
- Logging
- Monitoring
- Backups
- Deployment
- Privacy
- Legal implementation

Do not force every item into the project.

For example, a simple marketing website does not need a complex caching layer, multi tenant architecture, background job system or advanced authentication system.

Use the simplest architecture that safely satisfies the actual requirements.

Never replace working technology merely because another technology is more popular.

Never add a dependency without a reason.

Never trust frontend only validation.

Never trust frontend only authorization.

Never trust client supplied roles, permissions, prices, payment states or ownership.

Enforce security on the server.

For multi tenant applications, make sure tenant boundaries are enforced server side.

For authentication, use the authentication architecture already chosen unless there is a concrete reason to improve it.

If JWT is used, verify its signature, expiration and intended use correctly.

If sessions and cookies are used, verify secure cookie configuration and session lifecycle.

For APIs, validate input, enforce authorization, handle errors safely, use reasonable rate limits and prevent excessive resource usage.

For caching, verify that private data cannot leak through shared caches.

For file uploads, validate type and size and make sure users cannot access files they are not allowed to access.

For external services, handle failures, timeouts and retries carefully.

For payments and webhooks, verify events server side and make processing safe against duplicate delivery.

For SEO, make public pages crawlable and properly configured without accidentally exposing private application pages.

For performance, optimize actual bottlenecks rather than performing unnecessary micro optimization.

For legal and privacy, do not invent laws or legal conclusions. Identify the product behavior and areas that need jurisdiction specific review.

Make changes incrementally.

After every major change, verify that the affected functionality still works.

Run appropriate tests, type checks, lint checks and production builds when available.

Test important negative cases, not only successful cases.

After implementation, inspect the final diff.

Remove only code that you can confidently identify as unused or unnecessary.

Do not break unrelated functionality.

Before declaring the work complete, perform a final review covering:

- Functionality
- Security
- Authentication
- Authorization
- Database
- API
- Rate limiting
- Caching
- Performance
- SEO
- Accessibility
- Analytics
- Reliability
- Deployment
- Privacy
- Legal consistency

Return a final report with:

- What was changed
- What was not changed
- Files affected
- Tests performed
- Problems found
- Problems fixed
- Remaining risks
- Deployment requirements

Never claim that a check passed unless it was actually verified.

MASTER PROMPT — FINAL AUDIT

Use this after the project is considered finished.

Act as the final pre launch reviewer for this web application.

Do not rebuild the project.

Inspect the final implementation and verify that the application is actually ready for release.

Check the most important user flows first.

Then check:

- UI and responsive behavior
- Authentication
- Authorization
- API security
- Input validation
- Rate limiting
- Caching
- CDN and assets
- Database
- Performance
- SEO
- Accessibility

Analytics
Email
Payments if present
Webhooks if present
Logging
Monitoring
Backups
Environment variables
Deployment
Privacy and legal consistency

Look specifically for things that AI generated code commonly forgets.

Look for:

Missing authorization
Exposed private data
Incorrect tenant isolation
Missing rate limits
Unsafe file uploads
Leaked secrets
Broken environment configuration
Bad error handling
Missing loading and empty states
Broken mobile layouts
Missing SEO metadata
Missing sitemap
Wrong canonical URLs
Accidental indexing of private routes
Slow queries
Duplicate logic
Unnecessary dependencies
Broken production configuration

Do not create unnecessary architecture changes.

Classify findings as:

BLOCKER
HIGH
MEDIUM
LOW

Fix only problems that are safe to fix without changing unrelated behavior.

Then run the relevant tests and production build.

Finally report whether the project is:

READY

READY WITH MINOR RISKS

NOT READY

Explain the evidence for the result.

Simple Rule to Remember

The goal of vibe coding is not:

"AI wrote a lot of code."

The goal is:

"AI understood the existing system, changed only what was needed, tested it, and did not leave important production details forgotten."

Build the feature.

Secure the feature.

Make it fast enough.

Make it usable.

Make it discoverable.

Make it observable.

Make the legal and privacy behavior match reality.

Then launch.